

CS 382: Network-Centric Computing

Flow Control and Congestion Control



Dr. Zafar Ayyub Qazi

Momina Khan

Spring 2025

Agenda ...

- One more TCP implementation detail
 - How it implements checksums?
- How does the TCP sender adapt its window size?
 - Flow Control
 - Congestion Control

Assignment code will be tested in the exams

Where we are in the network stack?

Applications

...built on...

Reliable (or unreliable) transport

...built on...

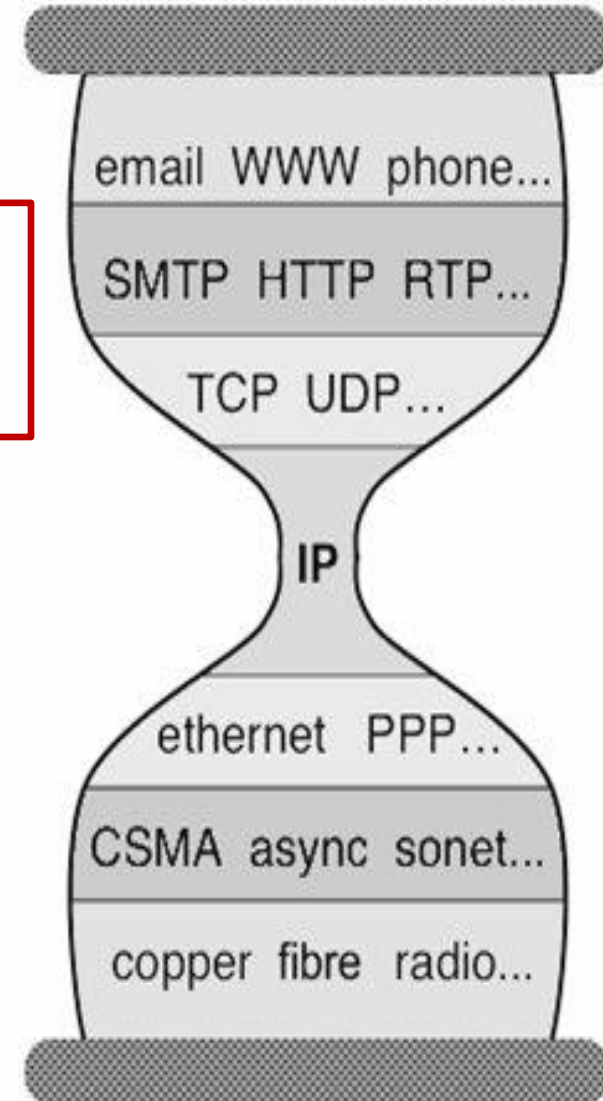
Best-effort global packet delivery

...built on...

Best-effort local packet delivery

...built on...

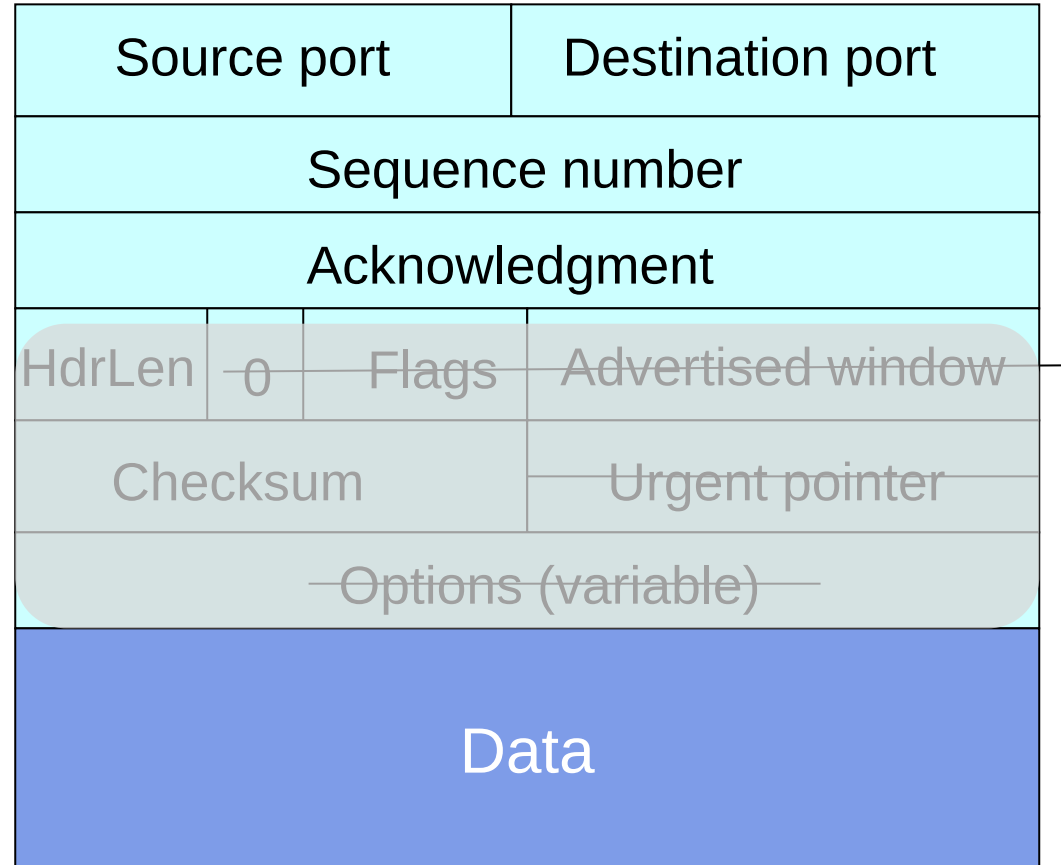
Physical transfer of bits



Transport Layer Tasks

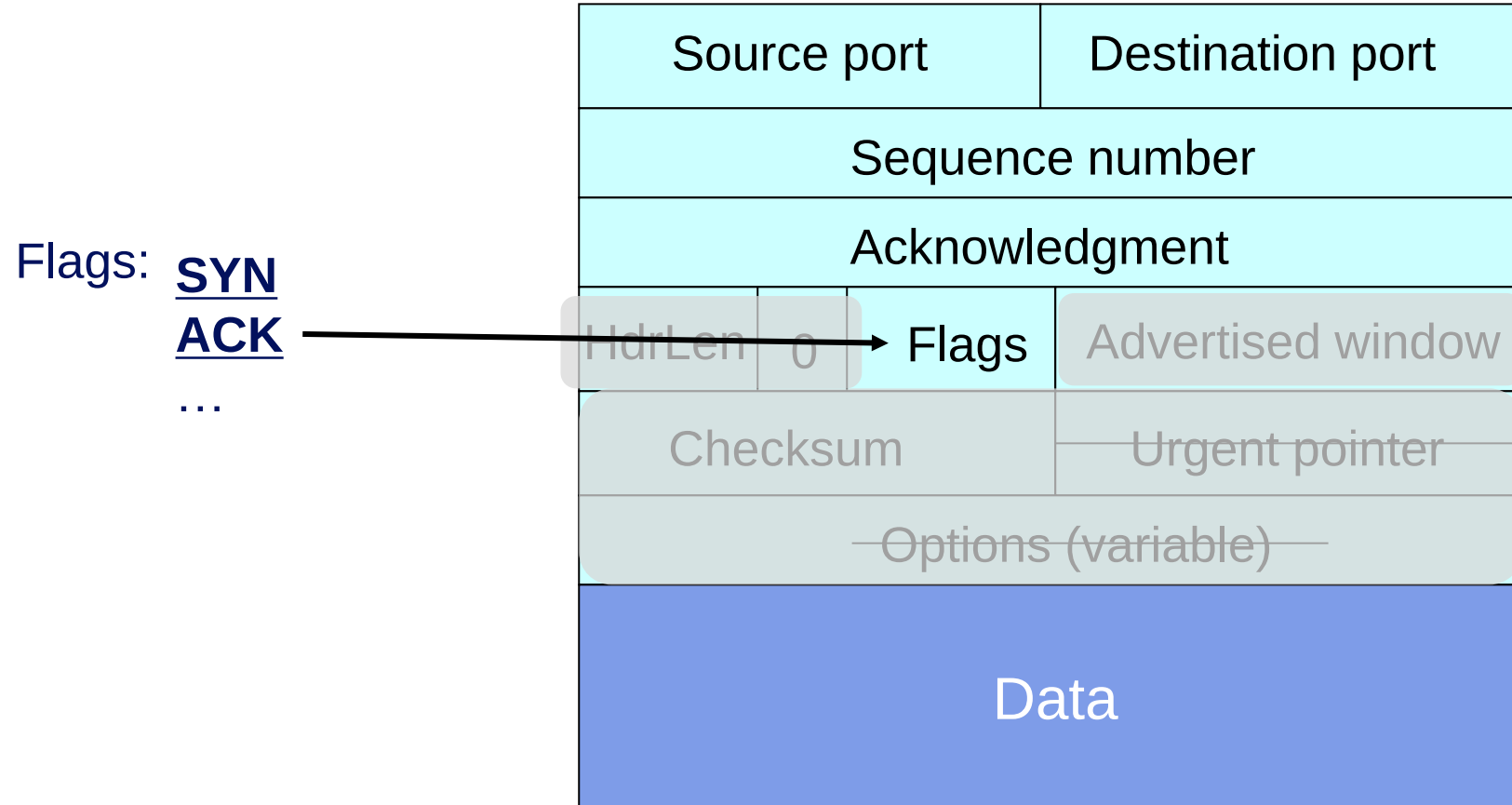
- Demultiplexing incoming data
 - Using ports
- Translating between bytestreams and packets
 - Segmentation and reassembly
- Reliability
 - ACKs, timeouts, retransmissions, etc.
- Identifying Packet corruption
 - Checksum
- Flow Control
 - Limit window size
- Congestion Control

TCP Header



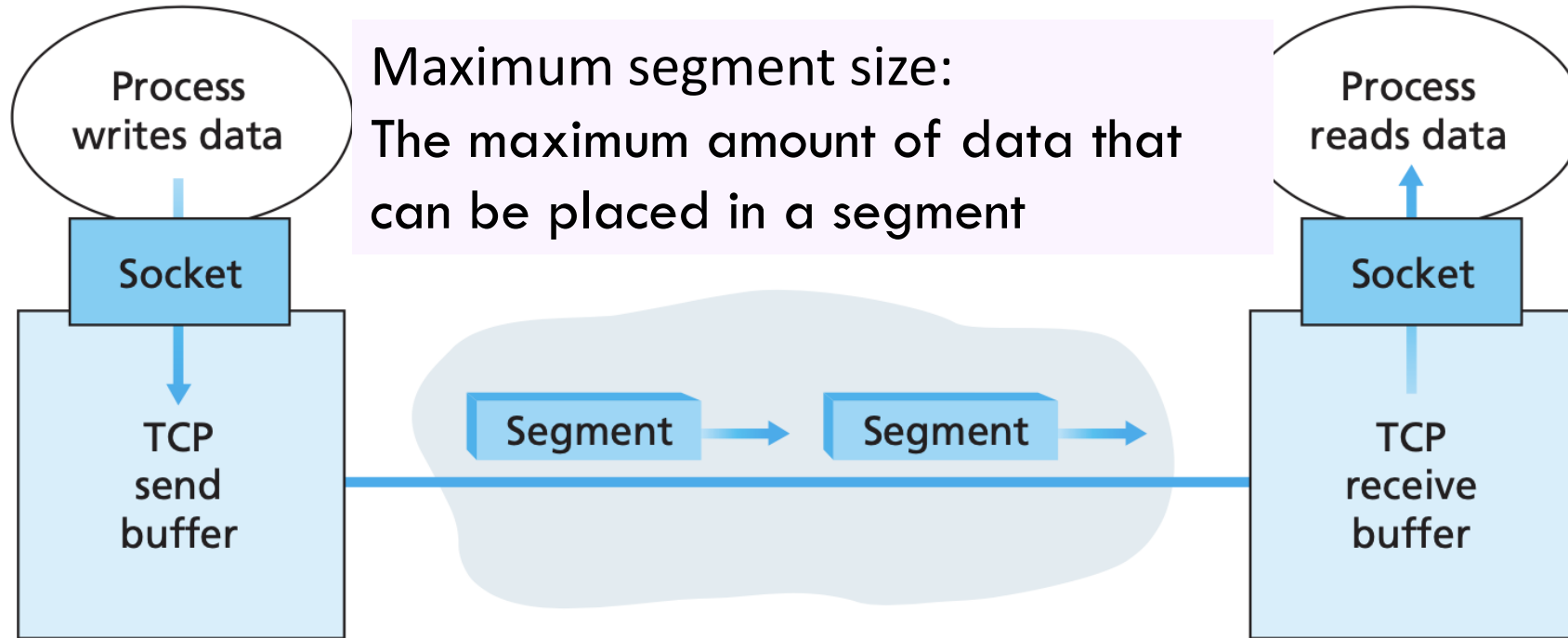
See `/usr/include/netinet/tcp.h` on Unix Systems

TCP Header



See /usr/include/netinet/tcp.h on Unix Systems

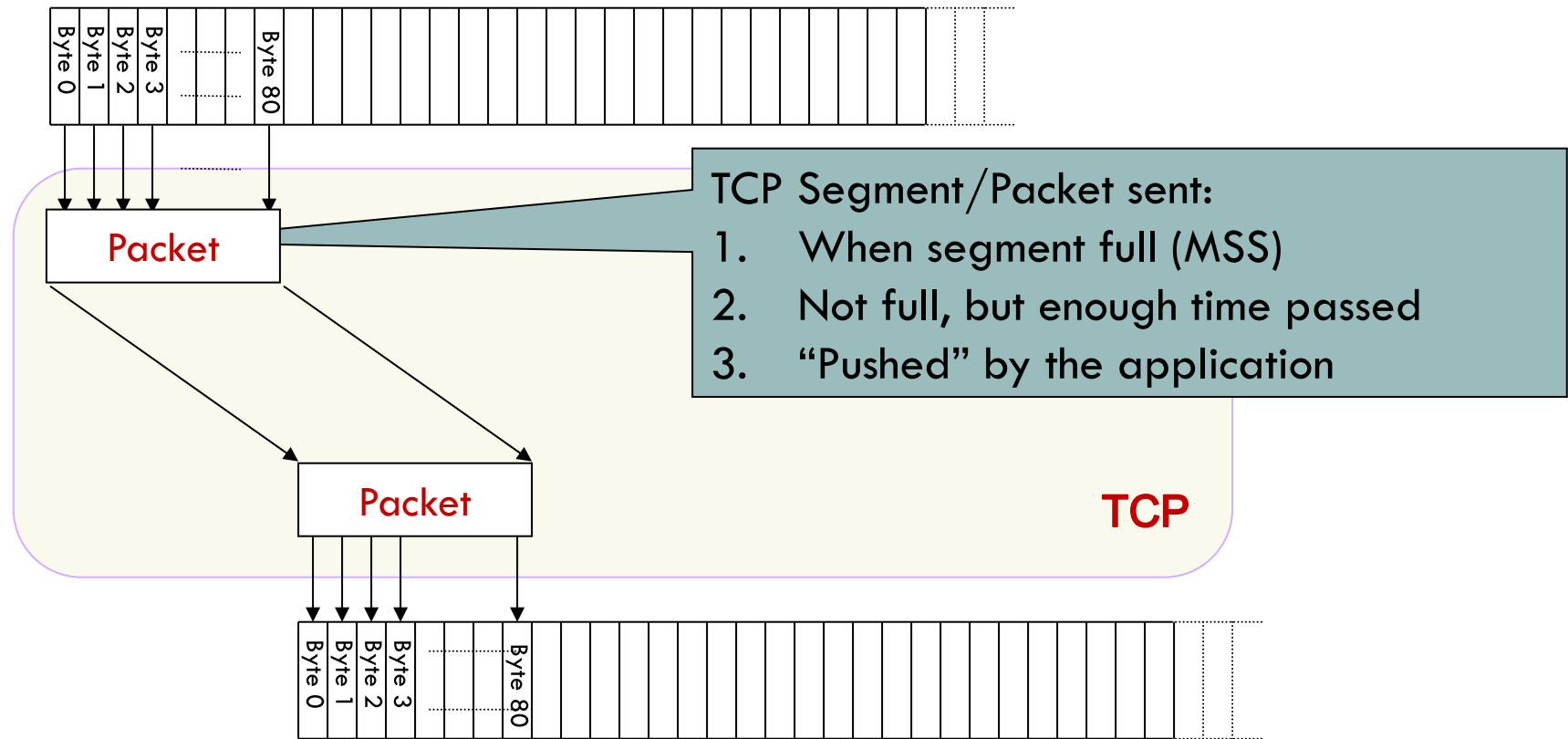
TCP Connections



Each side of the connection has its own send and receive buffers

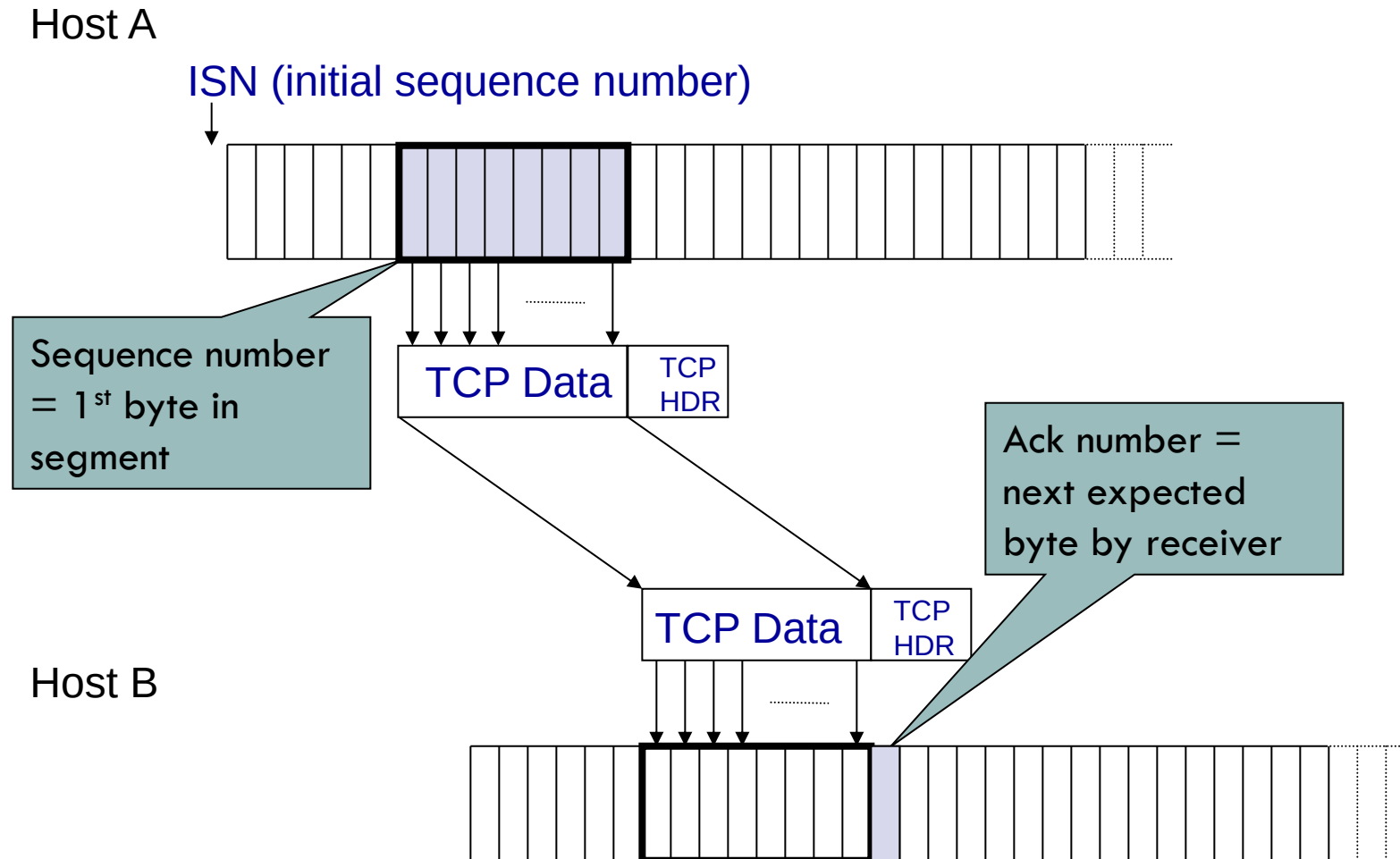
TCP Segmentation and Reassembly

Host A



Host B

Sequence numbers



TCP Checksum

Goal: detect “errors” (e.g., flipped bits) in transmitted segment
(e.g., due to noise in links, while stored in a router)

Sender:

- Treat segment contents as sequence of 16-bit words
- Checksum:
 - 1s complement of the sum of all the 16-bit words
- Sender puts checksum value into TCP checksum field

Receiver:

- Compute checksum of received segment
- Check if computed checksum equals checksum field value:
 - NO - error detected
 - YES - no error detected. *But maybe errors nonetheless. More later*

TCP Checksum Example

- Example: add two 16-bit integers
- Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

Read Book Chapter
3.3.2

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

TCP Checksum: Weak Protection

Example: add two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Even though numbers have changed (bit flips), **no** change in checksum!

There won't be any change in the checksum after we have flipped these four bits

Then Why is it Used?

- The reason for using an algorithm like this in spite of its weak protection against errors is
- One, this algorithm is **very easy to implement in software**
- Two, experience in the early Internet suggested that a **checksum of this form was adequate**
 - One reason it is adequate is that this checksum is the last line of defense in an end-end protocol
 - The majority of errors are picked by stronger error detection algorithms that run at the data link layer

Next: Flow Control

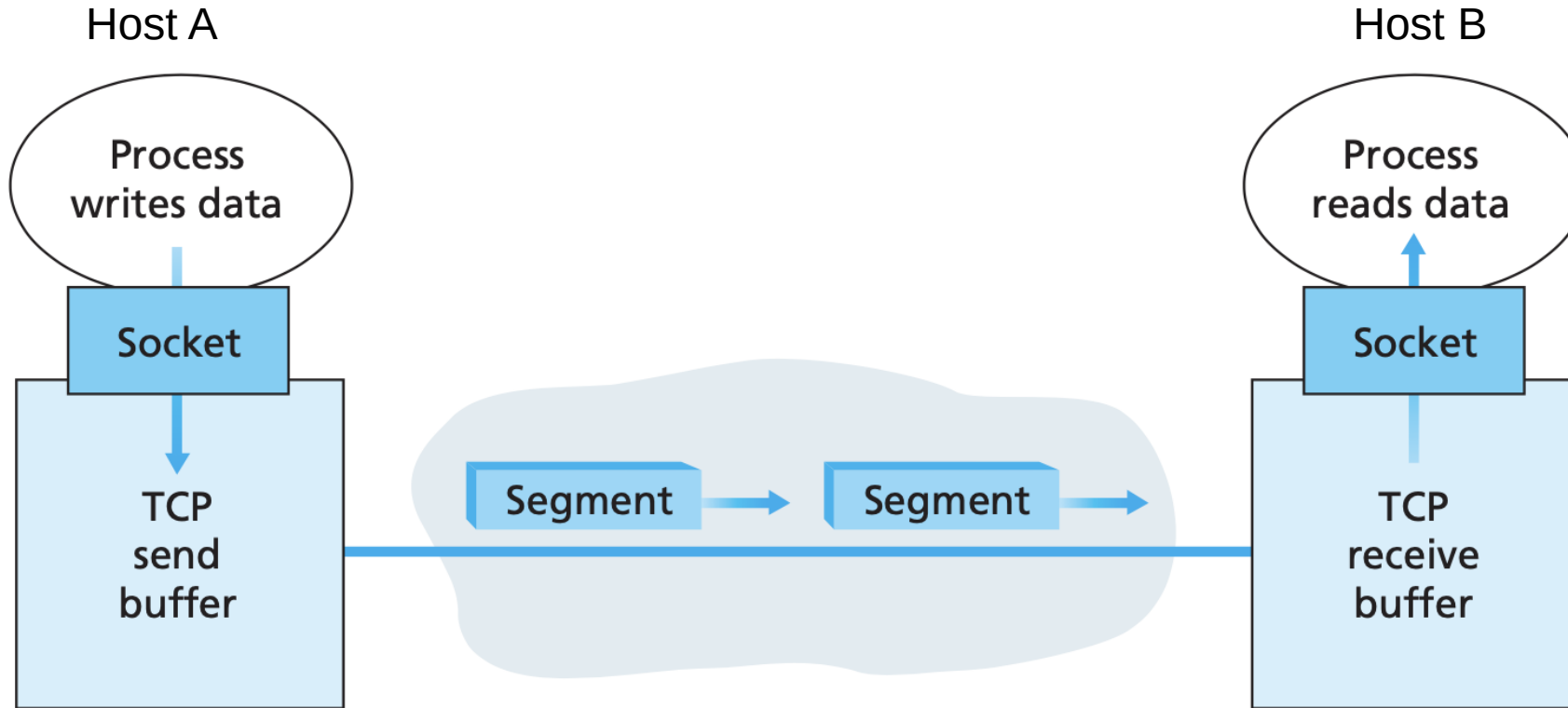
Flow Control

- Flow control keeps one fast sender from overwhelming a slow receiver

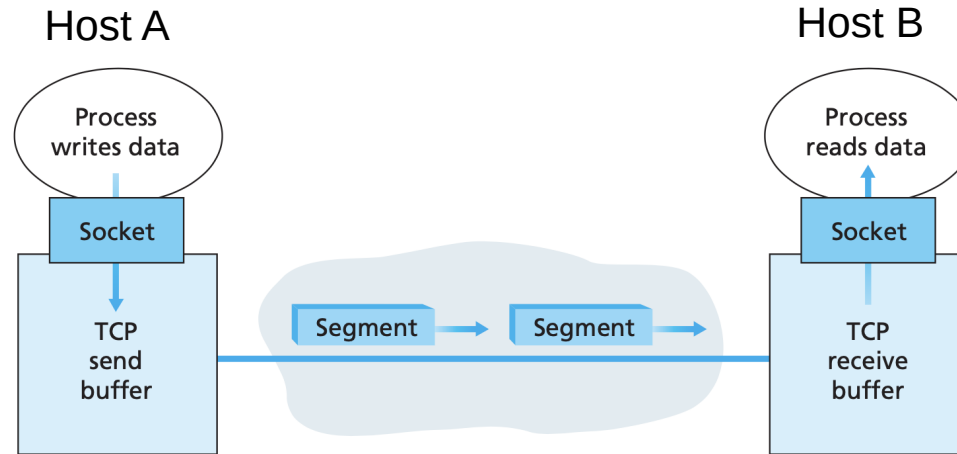
TCP Flow Control

- Sender maintains a variable called **receive window**
 - Based on the **Advertised Window** in the TCP header
- Receive window gives an idea **of how much free buffer space is available** at the receiver

TCP Flow Control Example



TCP Flow Control Example



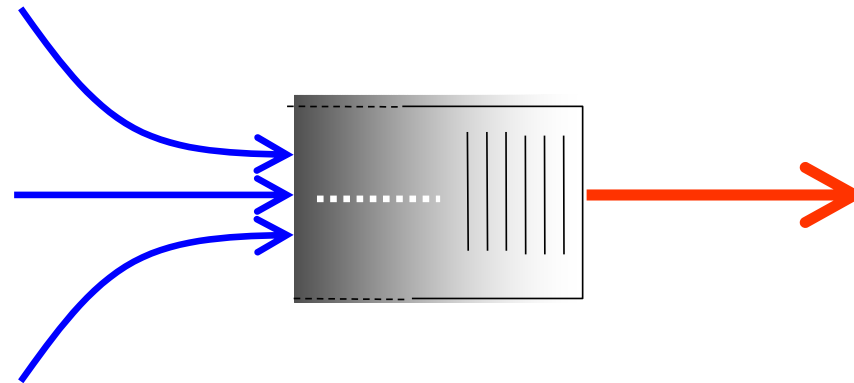
- **RcvBuffer:** TCP receive buffer size at Host B
- **LastByteRead:** number of last byte read by the process in B
- **LastByteRcvd:** number of last byte that has arrived from network in B
- Sender uses this advertised window to make sure that number of “in-flight” bytes are less than advertised window
- Advertised window sent by the receiver is:
 - $RcvBuffer - [LastByteRcvd - LastByteRead]$

Discussion

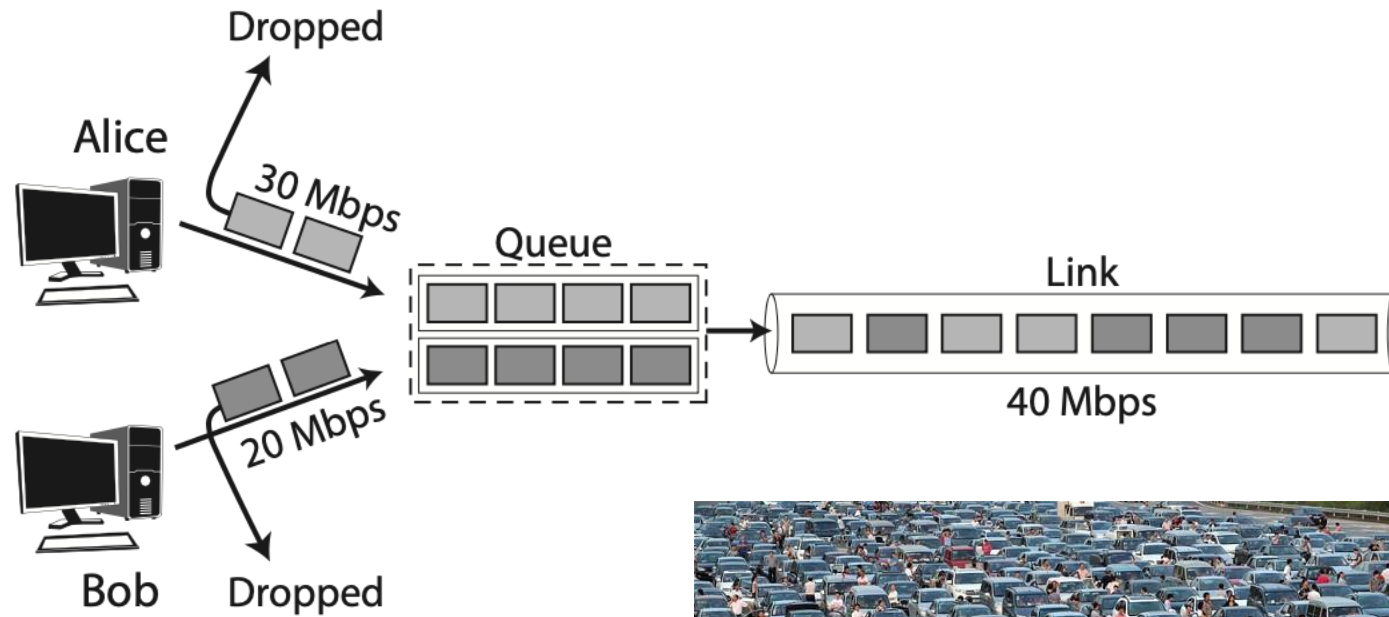
- Suppose the TCP sender decides the window size based on the following rule:
 - Window size: $\min(B \cdot \text{RTT}, \text{Advertised-Window})$
- What can go wrong?

Why congestion happens on the Internet?

- If two packets arrive at a router at the same time, the router will transmit one and buffer the other
- If many packets arrive close in time
 - the router cannot keep up → gets congested
 - causes packet delays and drops



Example Scenario



Similar to traffic
congestion on roads



Who Should Take Care of Congestion?

- Network?
- End hosts?
- Both?

Congestion Problem

- **Congestion Problem:** how do we avoid a set of senders from overloading the network?
- **Challenges:**
 - Senders are **unaware of each other**
 - Senders are **unaware of bottleneck link capacity**

Goals for Congestion Control

- Prevent or avoid congestion
 - Low packet delay and loss
- High link utilization
- Fair sharing of network resources

Lets go back in time; when first congestion collapse happened on the Internet.

First Congestion Collapse

- Sending rate only limited by flow control
 - Dropped packets → senders (repeatedly!) retransmit
- Led to “congestion collapse” in Oct 1986

In October of '86, the Internet had the first of what became a series of ‘congestion collapses’. During this period, the data throughput from LBL to UC Berkeley (sites separated by 400 yards and two IMP hops) dropped from 32 Kbps to 40 bps. We were fascinated by this sudden factor-of-thousand drop in bandwidth and embarked on an investigation of why things had gotten so bad. In particular, we wondered if the 4.3BSD (Berkeley UNIX) TCP was mis-behaving or if it could be tuned to work better under abysmal network conditions. The answer to both of these questions was “yes”.

Saving the Internet ...

- “Congestion Avoidance and Control” by Van Jacobson et al.
 - ACM SIGCOMM 1988
 - One of the most widely cited papers in CS
 - > 9000 citations
- Tremendous impact
- TCP implementations use several algorithms proposed in this work

Van Jacobson

- Researcher in the networking group at Lawrence Berkeley National Labs (LBNL)
- Many contributions to the early TCP/IP stack
- Creator of many widely used network tools
 - Traceroute, tcpdump, Berkeley Packet Filter
- Later Chief Scientist at Cisco, now at Google
 - Recently: BBR, a new TCP Congestion control protocol used by Google



Jacobson's Approach

- Extend TCP's existing window-based protocol but adapt the window size in response to congestion
- A pragmatic and effective solution
 - Required no upgrades to routers or applications!
 - Patch of a few lines of code to TCP implementations
- Extensively researched and improved upon
 - Countless variants (we will not discuss)
 - As well as different approaches, especially for datacenters and cloud services

Congestion Control

- Huge literature on the problem
 - In systems, game theory, stats, econ
- Recent resurgence of interest in industry
 - New context (datacenters, new app workloads)
 - New pressure for high-performance (cloud services)
 - New methods (ML)

Let's design a solution for controlling congestion



Building Blocks for a Solution

- What are the building blocks needed to design a congestion control algorithm?



Building Blocks for Congestion Control

- Discovering an initial rate
- Detecting congestion
- Reacting to congestion (or lack thereof)
 - Increase/decrease rules

Detecting Congestion

How can senders detect congestion?

- Network could tell sender about congestion
- Senders infer congestion when a packet is lost
- Senders infer congestion from packet delays